

Go! Férias — Stack Técnica

Documento de referência · Gerado em 14/08/2026

Node.js 20

Express 4

Prisma 5

PostgreSQL

JWT

Render

Backend oficial — `server/`

Runtime	Node.js 20
Framework	Express 4 (API REST)
ORM / Banco	Prisma 5 sobre PostgreSQL
Autenticação	JWT (<code>jsonwebtoken</code>) + senhas com <code>bcryptjs</code>
E-mail	Nodemailer (SMTP)
Utilitários	<code>cors</code> , <code>dotenv</code>

Estrutura em `src/` : `auth.js` , `mailer.js` , `audit.js` , `prisma.js` , `serialize.js` e a pasta `routes/` .
Schema e seed em `prisma/` .

Frontend

- **index.html único** (~1.9 MB) — SPA em JavaScript "vanilla", servida pelo próprio servidor Node.
- Semântica de cores do produto: **verde** = folga, **vermelho** = férias.

Importação de dados — `backend/import/`

- **Python** — migração da planilha dos ~120 funcionários reais.
- Scripts: `importar_form18.py` , `enviar_para_api.py` , `corrigir_setor.py` , `excluir_colab.py` , `importar_planilha.py` .

Deploy — `render.yaml`

- **Render** (Blueprint) — sobe API Node + frontend + PostgreSQL num único projeto.
- Fluxo de start: `prisma db push` → seed idempotente → `node src/index.js` .
- Health check em `/health` ; `DATABASE_URL` e `JWT_SECRET` injetados automaticamente.

Referência (preterido)

- `backend/` — implementação antiga com **PostgREST**, mantida apenas como referência histórica. Não é o backend ativo.

Resumo: Node + Express + Prisma + PostgreSQL no backend, HTML puro no frontend, Python só para importação, deploy no Render.